



Evaluación Parcial 1

Ejecución práctica

Instrucciones y pauta de evaluación | Estudiante

Evaluación Parcial 1

Ejecución práctica

Instrucciones y pauta de evaluación

Estudiante

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación	Semana inicio EA1
DSY1102	Programación Orientada a Objetos	3 horas	<u>25</u> %	Semana 1 / EA1

Instrucciones generales Evaluación Parcial 1

Descripción

La Evaluación Parcial 1 consiste en el desarrollo de un programa de consola funcional que resuelva el caso de negocio asignado por el/la docente.

El proyecto integrará los aprendizajes de la Experiencia de Aprendizaje 1, incluyendo la identificación del paradigma orientado a objetos, la implementación de estructuras de control y métodos, la construcción de clases con encapsulamiento, la aplicación de herencia y polimorfismo, el manejo de excepciones y el uso de colecciones.

Se entregará un caso de negocio al inicio de la sesión. El/la estudiante dispondrá del tiempo asignado para desarrollar la solución en el entorno de trabajo configurado en el laboratorio.

La evaluación debe ser desarrollada por el estudiante en el laboratorio de clases (TAITE9) durante la sesión asignada.

La evaluación se implementa en la semana 6. El trabajo es de carácter individual y su calificación se realizará en base al cumplimiento de los requisitos establecidos en el caso de negocio.

Los indicadores de logro evaluados son:

IL 1.1 Identifica las características del paradigma orientado a objetos respecto a otros paradigmas de programación, tales como, diferencias en tipo, estructura y ciclo de compilación, para contextualizar el uso del lenguaje en el desarrollo de software.

IL 1.2 Implementa estructuras de control condicionales y cíclicas para representar la lógica de solución de un caso de negocio, definiendo métodos con parámetros, retorno y sobrecarga para modularizar el comportamiento de la aplicación.

IL 1.3 Construye clases con atributos, constructores y métodos de acceso, aplicando encapsulamiento y colaboración entre objetos para modelar entidades interrelacionadas de un problema dado.

IL 1.4 Aplica herencia, clases abstractas e interfaces para definir jerarquías de tipos y especializar el comportamiento de clases derivadas en la solución de un caso de negocio.

IL 1.5 Aplica polimorfismo y manejo de excepciones para operar sobre conjuntos de objetos de distintos subtipos, garantizando la robustez de la aplicación ante errores en tiempo de ejecución.

IL 1.6 Emplea colecciones para almacenar, iterar y gestionar conjuntos de datos de forma temporal, seleccionando la estructura adecuada según las necesidades del caso de negocio.

Instrucciones específicas de la Evaluación:

Consideraciones para la realización de la ejecución práctica:

- Se recomienda realizar esta evaluación en sala de laboratorio equipada con computadores con acceso a un entorno de desarrollo Java (BlueJ o IntelliJ IDEA Community).
- No se permite el acceso a internet ni el uso de dispositivos inteligentes (teléfonos, relojes u otros) durante la evaluación.
- El/la docente debe monitorear activamente el trabajo del estudiante durante toda la sesión.

Requisitos para el desarrollo de la solución:

- El programa debe compilar y ejecutarse correctamente. Una solución que no compila no permite evidenciar los indicadores de logro.
- El código debe organizarse en clases separadas, aplicando los principios del paradigma orientado a objetos conforme a los requerimientos del caso de negocio.
- Deben incluirse comentarios en el código fuente que distingan el paradigma orientado a objetos de otros paradigmas de programación, según lo requerido en IL 1.1.

- La solución debe incluir manejo de excepciones para entradas inválidas del usuario, garantizando que el programa no se interrumpa ante errores de ingreso de datos.
- La jerarquía de clases, la interfaz, el uso de colecciones y el comportamiento polimórfico deben estar presentes y ser evaluables en el código entregado.

Forma de entrega:

- El proyecto debe comprimirse en formato ZIP e incluir todo el código fuente desarrollado durante la sesión.
- El archivo comprimido debe subirse a la plataforma AVA antes de finalizar la evaluación, utilizando la actividad de entrega habilitada para ese fin.
- No se aceptará entrega por ningún otro medio ni fuera del horario establecido. La ausencia de entrega en AVA generará la nota mínima de manera automática.

Consideraciones para todos los estudiantes:

- No se admite el uso de herramientas de inteligencia artificial ni de recursos digitales de apoyo externos.
- Cualquier indicio de colaboración no autorizada, uso de IA o plagio implicará la anulación inmediata de la evaluación y penalizado automáticamente con nota mínima de 1.0.
- Ante cualquier situación irregular observada, el/la docente tiene la facultad de dejar sin efecto la evaluación del estudiante involucrado.

Pauta de Evaluación (Rúbrica)

Categoría	% logro	Descripción niveles de logro
Muy buen desempeño	100%	Demuestras dominio sólido del indicador, implementación completa, coherencia técnica, estabilidad funcional y presentación profesional.
Buen desempeño	80%	Demuestras alto nivel de logro con pequeñas omisiones o detalles menores que no afectan la comprensión global de la solución.
Desempeño aceptable	60%	Cumples lo esencial del indicador, pero con limitaciones técnicas, organización básica o validaciones parciales.
Desempeño incipiente	30%	Presentas avances parciales, errores importantes o integración incompleta de componentes, navegación o persistencia.
Desempeño no logrado	0%	No evidencias el logro del indicador o la solución es incorrecta, incompleta o no funcional.

Indicador de Evaluación	Categorías de Respuesta					Ponderación Indicador de Evaluación
	Muy buen desempeño 100%	Buen desempeño 80%	Desempeño aceptable 60%	Desempeño incipiente 30%	Desempeño no logrado 0%	
IE 1.1.1 Distingue, mediante comentarios en el código fuente, al menos dos diferencias entre el lenguaje orientado a objetos utilizado y otro paradigma de programación, considerando tipado, declaración de variables y ciclo de ejecución.	Identifica con precisión dos o más diferencias entre paradigmas (tipado, declaración de variables y ciclo de ejecución), redactadas en palabras propias, sin errores conceptuales y directamente en el código fuente.	Identifica dos diferencias relevantes, pero una presenta una imprecisión conceptual menor que no afecta la comprensión general.	Identifica al menos una diferencia correcta, pero omite aspectos solicitados o presenta confusiones parciales entre los conceptos de ambos paradigmas.	Incluye comentarios que intentan comparar paradigmas, pero con errores conceptuales importantes o sin abordar los aspectos de tipado, declaración y ciclo de ejecución.	No incluye comentarios comparativos en el código fuente, o el contenido es incorrecto y no permite evidenciar distinción entre paradigmas.	4%
IE 1.1.2 Argumenta, en los comentarios del código fuente, cómo el tipado del lenguaje seleccionado favorece la detección temprana de errores durante el desarrollo de la aplicación.	Argumenta con claridad y precisión cómo el tipado estático detecta errores en compilación, con justificaciones coherentes con el lenguaje utilizado, sin errores conceptuales.	Argumenta correctamente la relación entre tipado y detección de errores, pero la justificación es parcial o le faltan argumentos que respalden el razonamiento.	Menciona la relación entre tipado y errores, pero la argumentación es superficial o confunde la detección en compilación con la detección en ejecución.	Incluye comentarios sobre el tipado, pero sin argumentar su relación con la detección temprana de errores o con errores conceptuales que distorsionan la idea.	No incluye argumentación sobre el tipado en el código fuente, o el contenido es ausente e incorrecto.	4%
IE 1.2.1 Implementa estructuras condicionales que representan correctamente las reglas de negocio del problema planteado, evaluando todas las condiciones requeridas sin omisiones ni redundancias.	Implementa todas las estructuras condicionales requeridas con condiciones exactas, sin omisiones ni redundancias, y la lógica produce resultados correctos para todos los casos del problema.	Implementa la mayoría de las condiciones correctamente, con alguna omisión menor o redundancia que no afecta el resultado principal de la solución.	Implementa algunas estructuras condicionales, pero omite casos relevantes del negocio o presenta condiciones incorrectas que afectan parcialmente el resultado.	Implementa estructuras condicionales con errores importantes en la lógica o cubre solo una parte mínima de los casos, sin representar adecuadamente las reglas de negocio.	No implementa estructuras condicionales o las existentes son incorrectas e impiden evaluar la representación de las reglas de negocio.	4%

IE 1.2.2 Define métodos con parámetros de entrada y valor de retorno explícito que modularicen el comportamiento de la solución, respetando las convenciones de nomenclatura del lenguaje.	Define métodos con parámetros y retorno explícito que modularicen correctamente el comportamiento, con nombres descriptivos en camelCase y sin redundancias ni errores de firma.	Define métodos con parámetros y retorno, pero con pequeñas inconsistencias en la nomenclatura o con algún método que podría modularizarse mejor sin afectar la funcionalidad general.	Define algunos métodos con parámetros y retorno, pero omite modularizar comportamientos relevantes o presenta errores en la firma que afectan parcialmente la claridad del código.	Define métodos con errores en los parámetros o retorno, o la modularización es mínima y no representa adecuadamente el comportamiento esperado de la solución.	No define métodos con parámetros y retorno explícito, o los existentes son incorrectos e impiden evaluar la modularización del comportamiento.	4%
IE 1.2.3 Aplica sobrecarga de métodos para representar distintas variantes de un mismo comportamiento, con firmas diferenciadas en tipo o cantidad de parámetros.	Aplica sobrecarga correctamente con al menos dos versiones del mismo método diferenciadas en tipo o cantidad de parámetros, ambas funcionales y coherentes.	Aplica sobrecarga con dos versiones del método, pero una presenta una inconsistencia menor en la firma o en la lógica interna que no afecta la compilación.	Intenta aplicar sobrecarga, pero las versiones del método no se diferencian correctamente en la firma o presentan errores que limitan su utilidad.	Declara métodos con el mismo nombre, pero sin diferenciar las firmas de forma válida para el lenguaje, generando errores o ambigüedad en la resolución.	No aplica sobrecarga de métodos o los métodos declarados no corresponden a variantes del mismo comportamiento.	2%
IE 1.3.1 Declara los atributos de las clases con el tipo de dato y modificador de acceso adecuados, respetando el principio de encapsulamiento y las convenciones del lenguaje.	Declara todos los atributos con el tipo de dato correcto y el modificador de acceso adecuado (private/protected), respetando el encapsulamiento y las convenciones del lenguaje sin excepción.	Declara la mayoría de los atributos correctamente, con alguna inconsistencia menor en el modificador de acceso o en el tipo de dato que no compromete el encapsulamiento general.	Declara algunos atributos correctamente, pero omite encapsular varios o usa tipos de dato inadecuados que afectan la integridad del modelo de clases.	Declara atributos con errores importantes en el modificador de acceso o en el tipo de dato, comprometiendo el encapsulamiento en la mayoría de las clases.	No declara atributos en las clases o los declarados son incorrectos e impiden evaluar el encapsulamiento.	4%
IE 1.3.2 Implementa constructores que inicializan correctamente todos los atributos de la clase, permitiendo la creación coherente	Implementa constructores en todas las clases que inicializan correctamente	Implementa constructores en la mayoría de las clases con inicialización	Implementa constructores en algunas clases, pero omite atributos	Implementa constructores con errores importantes en la inicialización o	No implementa constructores o los existentes no inicializan atributos,	4%

de objetos a partir de los requerimientos del problema.	todos los atributos, con firma coherente con los requerimientos y sin errores de asignación.	correcta, pero alguna clase omite inicializar un atributo o presenta una inconsistencia menor en la firma.	relevantes en la inicialización o los argumentos no corresponden a los requerimientos del problema.	solo en algunas clases, lo que impide la creación coherente de objetos en la mayoría de los casos.	impidiendo la creación de objetos.	
IE 1.3.3 Define métodos de acceso (get/set) que controlan la lectura y escritura de los atributos, garantizando la integridad de los datos del objeto.	Define getters y setters para todos los atributos encapsulados, con nombres en convención correcta, retorno y parámetro adecuados, garantizando la integridad de los datos sin excepciones.	Define getters y setters para la mayoría de los atributos, con alguna omisión o inconsistencia menor en el nombre o el tipo que no compromete la integridad general del objeto.	Define métodos de acceso para algunos atributos, omitiendo varios o presentando errores en el tipo de retorno o en el parámetro que afectan la integridad de los datos.	Define métodos de acceso con errores importantes en la firma o solo para una minoría de atributos, comprometiendo el control sobre los datos del objeto.	No define métodos de acceso o los existentes son incorrectos e impiden evaluar el control sobre los atributos.	4%
IE 1.3.4 Implementa métodos de instancia que representan el comportamiento propio de cada entidad del problema, con lógica coherente respecto a los requerimientos descritos.	Implementa métodos de instancia en todas las clases que representan correctamente el comportamiento de cada entidad, con lógica coherente, sin errores y alineada a los requerimientos.	Implementa métodos de instancia en la mayoría de las clases con lógica correcta, pero algún método presenta una omisión o imprecisión menor que no afecta el comportamiento general.	Implementa métodos de instancia en algunas clases, pero con lógica parcialmente incorrecta o que no refleja adecuadamente el comportamiento esperado de la entidad.	Implementa métodos de instancia con errores importantes en la lógica o solo en algunas clases, sin representar adecuadamente el comportamiento de las entidades del problema.	No implementa métodos de instancia o los existentes son incorrectos e impiden evaluar el comportamiento de las entidades.	4%
IE 1.3.5 Establece colaboración entre objetos invocando métodos de unas clases desde otras, de acuerdo con las relaciones definidas en el modelo del problema.	Establece colaboración entre todos los objetos que el problema requiere, invocando métodos de forma correcta y coherente con las relaciones del	Establece colaboración entre la mayoría de los objetos requeridos, con alguna omisión menor o invocación con pequeños errores que no	Establece colaboración entre algunos objetos, pero omite relaciones relevantes del modelo o presenta errores en las invocaciones que	Intenta establecer colaboración, pero con errores importantes en la invocación de métodos o sin respetar las relaciones definidas en el	No establece colaboración entre objetos o las invocaciones son incorrectas e impiden evaluar las relaciones del modelo.	4%

	modelo, sin errores de referencia.	afectan el flujo general de la solución.	afectan parcialmente el comportamiento.	modelo del problema.		
IE 1.4.1 Construye una jerarquía de clases que modela correctamente la relación de generalización-especialización del problema, declarando la clase base con los atributos y comportamientos comunes a todos los tipos.	Construye una jerarquía correcta y completa: la clase base agrupa todos los atributos y comportamientos comunes, y las subclases heredan y especializan según los requerimientos, sin errores estructurales.	Construye una jerarquía correcta en su mayor parte, con alguna omisión menor en los atributos comunes de la clase base o en la relación de herencia de alguna subclase.	Construye una jerarquía parcialmente correcta: la clase base existe, pero omite atributos o comportamientos comunes relevantes, o la herencia no está completamente establecida.	Intenta construir una jerarquía, pero con errores importantes: atributos comunes en subclases, herencia mal declarada o clase base sin comportamientos compartidos.	No construye jerarquía de clases o la estructura no refleja la relación de generalización-especialización del problema.	7%
IE 1.4.2 Declara métodos abstractos en la clase base para los comportamientos que dependen exclusivamente del tipo concreto, forzando su implementación en cada subclase derivada.	Declara todos los métodos abstractos necesarios en la clase base, con firma correcta, sin cuerpo de implementación, forzando su implementación en cada subclase derivada sin excepciones.	Declara la mayoría de los métodos abstractos requeridos correctamente, con alguna omisión menor o un método con pequeñas inconsistencias en la firma.	Declara algunos métodos abstractos, pero omite los más relevantes para el problema o los declara con errores que no fuerzan correctamente la implementación en las subclases.	Intenta declarar métodos abstractos, pero con errores importantes como incluir cuerpo en la declaración, no marcarlos como abstractos o no ser implementados en las subclases.	No declara métodos abstractos en la clase base o la clase no está declarada como abstracta.	6%
IE 1.4.3 Implementa la interfaz requerida en la clase concreta que corresponda, definiendo en ella todos los métodos declarados por el contrato de manera consistente con el contexto del problema.	Implementa la interfaz correctamente en la clase concreta correspondiente, definiendo todos los métodos del contrato con lógica coherente y consistente con el contexto del problema.	Implementa la interfaz en la clase correcta con la mayoría de los métodos del contrato, pero uno presenta una omisión o implementación parcial que no afecta el comportamiento general.	Implementa la interfaz, pero omite algunos métodos del contrato o los implementa con lógica que no es coherente con el contexto del problema.	Intenta implementar la interfaz, pero con errores importantes: métodos ausentes, firma incorrecta o implementación en una clase que no corresponde según el problema.	No implementa la interfaz o la clase concreta no la declara, impidiendo evaluar el cumplimiento del contrato.	6%
IE 1.4.4 Especializa el comportamiento de cada subclase sobrescribiendo los métodos	Sobrescribe correctamente todos los métodos	Sobrescribe la mayoría de los métodos en las	Sobrescribe algunos métodos en las subclases,	Intenta sobrescribir métodos, pero	No sobrescribe métodos en las subclases o las	6%

heredados con la lógica particular que los requerimientos del problema establecen para ese tipo.	heredados que requieren especialización en cada subclase, con lógica exacta a los requerimientos del problema y uso apropiado de la anotación correspondiente.	subclases correctamente, con alguna omisión menor o una subclase con una ligera imprecisión en la lógica especializada.	pero omite especializar comportamientos relevantes o la lógica no refleja con precisión los requerimientos del tipo concreto.	con errores importantes en la lógica especializada o sin respetar la firma del método heredado en la mayoría de los casos.	implementaciones no especializan el comportamiento según los requerimientos del problema.	
IE 1.5.1 Opera sobre los objetos de la colección de forma polimórfica, invocando métodos de la clase base sin recurrir a verificaciones explícitas del tipo concreto de cada objeto.	Opera sobre todos los objetos de la colección de forma polimórfica, invocando métodos de la clase base en todos los casos, sin verificaciones explícitas del tipo concreto.	Opera de forma polimórfica en la mayoría de los casos, con alguna verificación de tipo puntual y justificada que no afecta el comportamiento polimórfico general de la solución.	Opera de forma polimórfica en algunos casos, pero recurre a verificaciones explícitas del tipo concreto en situaciones donde el polimorfismo debería ser suficiente.	Intenta operar sobre la colección, pero con verificaciones explícitas del tipo en la mayoría de los casos, sin aprovechar el polimorfismo como mecanismo de diseño.	No opera sobre objetos de forma polimórfica o el código no refleja el uso de referencias de la clase base para invocar métodos sobre objetos de distintos subtipos.	7%
IE 1.5.2 Implementa bloques de manejo de excepciones que capturan los errores producidos por entradas inválidas del usuario, informando el problema con un mensaje claro y retomando la solicitud del dato.	Implementa bloques try-catch en todos los puntos de entrada del usuario, captura las excepciones correspondientes, informa el error con un mensaje claro y retoma la solicitud del dato sin interrumpir la ejecución.	Implementa bloques try-catch en la mayoría de los puntos de entrada, con algún punto sin manejo o con un mensaje de error poco descriptivo que no afecta el flujo general.	Implementa bloques try-catch en algunos puntos de entrada, pero omite situaciones relevantes o el programa se interrumpe en algún caso ante una entrada inválida.	Implementa bloques try-catch de forma mínima o incorrecta, sin capturar las excepciones adecuadas o sin retomar la solicitud del dato tras el error.	No implementa manejo de excepciones o el programa se interrumpe ante cualquier entrada inválida del usuario.	7%
IE 1.5.3 Valida que los valores ingresados por el usuario cumplan las restricciones de negocio definidas (tipo de dato y rango válido) antes de utilizarlos en el procesamiento de la solución.	Valida todos los valores ingresados verificando tipo de dato y rango válido antes de procesarlos, con mensajes claros en cada caso y sin permitir que datos	Valida la mayoría de los valores ingresados, con alguna restricción de rango o tipo omitida que no afecta el procesamiento	Valida algunos valores ingresados, pero omite restricciones de negocio relevantes o permite que datos inválidos lleguen	Valida de forma mínima, verificando solo el tipo de dato sin considerar el rango válido, o la validación es inconsistente y no protege	No valida los valores ingresados por el usuario o los utiliza directamente sin verificar tipo ni rango.	5%

	inválidos lleguen al procesamiento.	principal de la solución.	al procesamiento en ciertos casos.	adecuadamente el procesamiento.		
IE 1.5.4 El programa no se interrumpe ante entradas inesperadas del usuario, manteniendo el flujo de ejecución de manera controlada hasta obtener un valor válido.	El programa mantiene el flujo de ejecución en todos los casos de entradas inesperadas, solicitando el dato nuevamente de forma controlada hasta obtener un valor válido, sin interrupción ni cierre inesperado.	El programa mantiene el flujo controlado en la mayoría de los casos, con algún escenario puntual en que la solicitud del dato no se reanuda correctamente.	El programa mantiene el flujo en algunos casos, pero se interrumpe o no solicita nuevamente el dato en ciertas situaciones de entrada inválida.	El programa intenta manejar entradas inesperadas, pero se interrumpe frecuentemente o no solicita el dato nuevamente tras el error en la mayoría de los casos.	El programa se interrumpe ante cualquier entrada inesperada del usuario, sin mantener el flujo de ejecución.	3%
IE 1.6.1 Declara una colección parametrizada con el tipo de la clase base de la jerarquía, permitiendo el almacenamiento uniforme de objetos de distintos subtipos en una misma estructura.	Declara una colección correctamente parametrizada con el tipo de la clase base, permitiendo almacenar objetos de todos los subtipos requeridos de forma uniforme y sin errores de tipo.	Declara la colección con el tipo correcto, con alguna inconsistencia menor en la parametrización o en el nombre de la variable que no afecta el almacenamiento de los objetos.	Declara la colección, pero con un tipo de parametrización inadecuado (tipo concreto en lugar de la clase base) que limita el almacenamiento uniforme de distintos subtipos.	Declara la colección sin parametrización correcta o con un tipo que no permite almacenar los subtipos requeridos, comprometiendo el diseño de la solución.	No declara una colección parametrizada o usa una estructura que no permite almacenamiento polimórfico de los objetos de la jerarquía.	5%
IE 1.6.2 Recorre la colección mediante una estructura iterativa que permite acceder y mostrar la información de cada objeto almacenado, aplicando el comportamiento polimórfico correspondiente.	Recorre la colección completa con una estructura iterativa correcta, accede a la información de cada objeto e invoca el método correspondiente de forma polimórfica en todos los casos.	Recorre la colección en la mayoría de los casos correctamente, con alguna omisión puntual en el acceso a la información de algún objeto o en la invocación polimórfica.	Recorre la colección, pero con errores en la estructura iterativa o sin invocar el comportamiento polimórfico en todos los objetos, afectando la visualización de la información.	Intenta recorrer la colección, pero con errores importantes en la iteración o sin acceder correctamente a la información de los objetos almacenados.	No recorre la colección o la estructura iterativa no permite acceder a los objetos y mostrar su información.	5%
IE 1.6.3 Gestiona el contenido de la colección aplicando al menos una operación de adición, consulta o filtrado de elementos, de acuerdo con las operaciones requeridas por el caso de negocio.	Implementa todas las operaciones de gestión requeridas (adición, consulta o filtrado) de forma correcta y	Implementa la mayoría de las operaciones requeridas correctamente, con alguna	Implementa al menos una operación de gestión correctamente, pero omite otras	Intenta implementar operaciones de gestión, pero con errores importantes que	No implementa operaciones de gestión sobre la colección o las existentes son incorrectas e	5%

	coherente con el caso de negocio, sin errores en la manipulación de la colección.	operación con una omisión menor o implementación parcial que no afecta la gestión general.	operaciones requeridas o las implementa con errores que afectan el resultado esperado.	no permiten manipular el contenido de la colección de acuerdo con los requerimientos del negocio.	impiden evaluar la manipulación de los datos.	
Total						100%